SQL Tutorial

https://www.w3schools.com/sql/

Tema	Notater
SQL Syntax	De fleste handlingene vi foretar oss i en database er gjort med SQL statements. For eksempel hvis vi skal hente ut all informasjon i en tabell som heter customers: SELECT * FROM Customers Obs: SQL er ikke case sensitive, men det er vanlig å skrive SQL i upper case. **Noen av de viktigste SQL kommandoene:** **SELECT** - Henter data fra en database **UPDATE** - Oppdaterer data i en database **DELETE** - Sletter data fra en database **INSERT INTO** - Setter inn ny data i en database **CREATE DATABASE** - Oppretter en ny database **ALTER DATABASE** - Endrer en database **CREATE TABLE** - Oppretter en ny tabell **ALTER TABLE** - Endrer en tabell **DROP TABLE** - Sletter en tabell **CREATE INDEX** - Oppretter en index (søkeord) **DROP INDEX** - Sletter en index
SQL SELECT Statement	SELECT statement er brukt til å velge ut data fra en database. Dataen som blir returnert lagres i en resultat tabell kalt result-set. **Syntaks:** **SELECT** rad1, rad2 **FROM** tabellnavn rad1 og rad2 er feltnavnene hvor man vil hente data fra. **SELECT * FROM** tabellnavn Henter alle tilgjengelige felter i tabellen
SQL SELECT DISTINCT Statement	**SELECT DISTINCT** brukes til å returnere utelukkende ulike verdier. I en tabell har ofte kolonner mange like verdier, så for å hente ut unike verdier kan man bruke denne kommandoen. **Syntaks:** **SELECT DISTINCT** kolonne1, kolonne2, ... **FROM** tabellnavn **SELECT DISTINCT** rad1 **FROM** tabellnavn Henter alle forskjellige verdier i rad1 fra tabellnavn **SELECT COUNT(DISTINCT** rad1**) FROM** tabellnavn

	Returnerer hvor mange ulike verdier man har i en tabell
SQL WHERE Clause	**WHERE** klausulen brukes til å filtrere oppføringer. Den brukes til å kun hente ut oppføringer som oppfyller en spesifisert betingelse. **Syntaks:** **SELECT** kolonne1, kolonne2, ... **FROM** tabellnavn **WHERE** betingelse OBS: **WHERE** klausulen brukes ikke bare med **SELECT** uttalelser, men brukes også med **UPDATE, DELETE,** etc. **SELECT * FROM** Customers **WHERE** Country='Mexico'; Denne henter ut alle kundene fra Mexico i Customers tabellen. **Text fields vs numeric fields** SQL krever single quotes rundt tekstverdier (de fleste databasesystemer støtter også vanlige quotes også). Tall derimot skal aldri være i quotes! **SELECT * FROM** Customers **WHERE** CustomerID=1; **Operatorer:** Disse operatorene kan brukes sammen med **WHERE** **=** er lik **>** større enn **<** mindre enn **>=** større enn eller lik **<=** mindre enn eller lik **<>** ikke lik - skrives noen ganger som != **BETWEEN** mellom et visst område **LIKE** søker etter et mønster **IN** for å spesifisere flere mulige verdier i en kolonne
SQL AND, OR and NOT Operators	**WHERE** kan kombineres med **AND, OR** og **NOT** operatorene. **AND** og **OR** brukes til å filtrere ut oppføringer etter mer enn en betingelse: – **AND** operatoren viser oppføringer hvor betingelsene separert med **AND** er **TRUE** – **OR** operatoren viser oppføringer hvor betingelsene separert med **OR** er **TRUE**. **NOT** brukes for å vise oppføringer hvor betingelsen(e) er **NOT TRUE**. **Syntaks:**

	SELECT kolonne1, kolonne2, ... **FROM** tabellnavn **WHERE** betingelse1 **AND** betingelse2 **AND** betingelse3; **SELECT** kolonne1, kolonne2, ... **FROM** tabellnavn **WHERE** betingelse1 **OR** betingelse2 **OR** betingelse3; **SELECT** kolonne1, kolonne2, ... **FROM** tabellnavn **WHERE NOT** betingelse; **Kombinere:** Man kan også kombinere disse operatorene. Feks: **SELECT * FROM** Customers **WHERE** Country='Germany' **AND** (City='Berlin' **OR** City='München'); Henter ut alle kunder fra Tyskland og som bor i Berlin eller i München. **SELECT * FROM** Customers **WHERE NOT** Country='Germany' **AND NOT** Country='USA'; Henter ut alle kunder som ikke er fra Tyskland og ikke fra USA.
SQL ORDER BY Keyword	**ORDER BY** nøkkelordet brukes til å sortere resultatene i stigende eller synkende rekkefølge. Resultatene blir sortert i stigende rekkefølge som standard. For å sortere etter synkende rekkefølge bruker man **DESC** **Syntaks:** **SELECT** column1, column2, ... **FROM** table_name **ORDER BY** column1, column2, ... **ASC|DESC;** Man kan sortere etter flere kolonner. For eksempel: **SELECT * FROM** Customers **ORDER BY** Country, CustomerName; I dette tilfellet vil det først sorteres stigende etter land, og hvis det er noen med likt land vil de deretter bli sortert etter kundens navn. Man kan også sortere etter flere kolonner, men den ene er stigende og den andre sykende. Feks: **SELECT * FROM** Customers **ORDER BY** Country **ASC**, CustomerName **DESC**;
SQL INSERT INTO Statement	**INSERT INTO** brukes for å sette inn nye poster i en tabell. **Syntaks:** Man kan skrive **INSERT INTO** på to forskjellige måter.

	– Spesifisere både kolonnenenavn og verdiene som skal settes inn: **INSERT INTO** table_name (column1, column2, column3, ...) **VALUES** (value1, value2, value3, ...); – Hvis man setter inn verdier for alle kolonnene i tabellen trenger man ikke spesifisere kolonnenavn. Man må huske på å sette inn verdiene i riktig rekkefølge så de havner i riktig kolonne. **INSERT INTO** table_name **VALUES** (value1, value2, value3, ...); Eks: **INSERT INTO** Customers (CustomerName, ContactName, Address, City, PostalCode, Country) **VALUES** ('Cardinal', 'Tom B. Erichsen', 'Skagen 21', 'Stavanger', '4006', 'Norway'); Man kan også sette inn data bare i spesifiserte kolonner, og ikke i alle.
SQL NULL VALUES	Et felt med en **NULL** verdi er et felt uten noen verdi. Hvis et felt i en tabell er valgfri så er det mulig å sette inn nye poster eller oppdatere poster uten å legge inn noen verdi i det feltet. Da lagres feltet med en **NULL** verdi. OBS: En **NULL** verdi er forskjellig fra 0 eller et blankt felt, da disse er faktiske verdier. Et felt med **NULL** er tomt fra posten ble opprettet. Man kan ikke bruke sammenlikningsoperatorer på **NULL** verdier, så man bruker **IS NULL** og **IS NOT NULL**. **Syntaks:** **SELECT** column_names **FROM** table_name **WHERE** column_name **IS NULL**; **SELECT** column_names **FROM** table_name **WHERE** column_name **IS NOT NULL**; **IS NULL** operatoren brukes til å sjekke for tomme verdier (NULL). Man får da resultat tilbake hvor det listes opp alle man har valgt for den valgte kolonnen er NULL. **IS NOT NULL** brukes da til det motsatte, altså å teste for ikke tomme verdier.
SQL UPDATE Statement	**UPDATE** uttalelsen brukes til å modifisere eksisterende felt i en tabell. **Syntaks:** **UPDATE** table_name **SET** column1 = value1, column2 = value2, ...

	WHERE condition; **OBS:** Det er viktig å ha med **WHERE** når man bruker **UPDATE**! Hvis man ikke har dette med så vil alle feltene i tabellen bli endret. **Eks:** **UPDATE** Customers **SET** ContactName = 'Alfred Schmidt', City= 'Frankfurt' **WHERE** CustomerID = 1; Ved hjelp av WHERE her så sørger man for at det *kun* er kunde 1 som blir oppdatert og ikke alle kunder. Vi kan endre flere felter ved å endre **WHERE** betingelsen. Feks: **UPDATE** Customers **SET** ContactName='Juan' **WHERE** Country='Mexico'; I dette eksempelet vil alle med Mexico i Country feltet få oppdatert navnet til Juan. Vi kan også ha flere betingelser i **SET** og skille med komma. Feks: **UPDATE** Customers **SET** City=’Oslo’, Country=’Norway’ **WHERE** CustomerID=32;
SQL DELETE Statement	**DELETE** uttalelsen brukes til å slette eksisterende poster i en tabell. **Syntaks:** **DELETE FROM** table_name **WHERE** condition; **OBS:** Det er viktig å bruke **WHERE** når vi sletter for å spesifisere hvilke poster som skal slettes. Uten denne vil alle postene i tabellen bli slettet. **Eks:** **DELETE FROM** Customers **WHERE** CustomerName='Alfreds Futterkiste'; Vi kan slette hele rader i en tabell uten å slette tabellen. Det betyr at tabellstrukturen, attributtene og indeksene vil være intakt. Feks: **DELETE FROM** Customers; Denne vil slette alle radene i Customers uten å slette tabellen.
SQL TOP, LIMIT, FETCH FIRST or ROWNUM Clause	**SELECT TOP** klausulen brukes til å spesifisere antall poster som skal returneres. Denne er nyttig for store tabeller med tusenvis av poster. Å hente ut store mengder poster kan gå utover ytelse. **OBS:** Ikke alle databasesystemer støtter **SELECT TOP**. MySQL støtter **LIMIT** for å velge ut antall, Oracle bruker **FETCH FIRST n ROWS ONLY** og **ROWNUM**

	<u>SQL Server / MS Access syntaks:</u> **SELECT TOP** number|**percent** column_name(s) **FROM** table_name **WHERE** condition; **<u>MySQL syntaks:</u>** **SELECT** column_name(s) **FROM** table_name **WHERE** condition **LIMIT** number; **<u>Oracle 12 syntaks:</u>** **SELECT** column_name(s) **FROM** table_name **ORDER BY** column_name(s) **FETCH FIRST** number **ROWS ONLY**; Vi kommer til å bruke MySQL syntaks. Eks: **SELECT * FROM** Customers **LIMIT** 3; SQL Server/MS Access og Oracle støtter å velge ut prosent. Feks SQL Server: **SELECT TO**P 50 **PERCENT * FROM** Customers; Vi kan bruke **WHERE** for å spesifisere hvor vi ønsker å velge ut fra i kombinasjon med dette.
SQL MIN() and MAX() Functions	**MIN()** funksjonen returnerer den minste verdien fra en valgt kolonne. **MAKS()** funksjonen returnerer den største verdien fra en valgt kolonne. **Syntaks:** **SELECT MIN(**column_name**)** **FROM** table_name **WHERE** condition; **SELECT MAX(**column_name**)** **FROM** table_name **WHERE** condition; **Eks:** **SELECT MIN(**Price**) AS** SmallestPrice **FROM** Products; Finner det billigste produktet og returnerer det som SmallestPrice **SELECT MAX(**Price**) AS** LargestPrice **FROM** Products; Finner det dyreste produktet og returnerer det som LargestPrice
SQL	**COUNT()** funksjonen returnerer antall rader som matcher det spesifiserte kriteriet.

COUNT(), AVG() and SUM() Functions	**SELECT COUNT(**column_name**)** **FROM** table_name **WHERE** condition; **AVG()** funksjonen returnerer gjennomsnittsverdien av en kolonne bestående av tall. **SELECT AVG(**column_name**)** **FROM** table_name **WHERE** condition; **SUM()** funksjonen returnerer den totale summen av en kolonne bestående av tall. **SELECT SUM(**column_name**)** **FROM** table_name **WHERE** condition; **OBS: NULL** verdier blir ikke medregnet i disse funksjonene!
SQL LIKE Operator	**LIKE** operatoren brukes i en **WHERE** for å søke etter spesifiserte mønster i en kolonne. Det er to wildcards som blir brukt sammen med **LIKE**: – (%) som representerer null, en eller flere tegn – (_) som representerer en enkelt tegn Disse kan også brukes i kombinasjon med hverandre. **Eksempler:** **WHERE** CustomerName **LIKE** 'a%' Finner verdier som starter med a. **WHERE** CustomerName **LIKE** '%a' Finner verdier som slutter med a **WHERE** CustomerName **LIKE** '%or%' Finner verdier som har or i hvilken som helst posisjon **WHERE** CustomerName **LIKE** '_r%' Finner verdier som har r som andre posisjon **WHERE** CustomerName **LIKE** 'a_%' Finner verdier som starter på a og har minst to tegn. **WHERE** CustomerName **LIKE** 'a__%' Finner verdier som starter med a og har minst tre tegn. **WHERE** ContactName **LIKE** 'a%o' Finner verdier som starter med a og slutter på o

LIKE kan også brukes sammen med operatorer som **NOT**.

SQL Wildcards

Et wildcard er en karakter som blir brukt til å substituere en eller flere tegn i en streng.

Wildcard Characters in MS Access

Symbol	Description	Example
*	Represents zero or more characters	bl* finds bl, black, blue, and blob
?	Represents a single character	h?t finds hot, hat, and hit
[]	Represents any single character within the brackets	h[oa]t finds hot and hat, but not hit
!	Represents any character not in the brackets	h[!oa]t finds hit, but not hot and hat
-	Represents any single character within the specified range	c[a-b]t finds cat and cbt
#	Represents any single numeric character	2#5 finds 205, 215, 225, 235, 245, 255, 265, 275, 285, and 295

Wildcard Characters in SQL Server

Symbol	Description	Example
%	Represents zero or more characters	bl% finds bl, black, blue, and blob
_	Represents a single character	h_t finds hot, hat, and hit
[]	Represents any single character within the brackets	h[oa]t finds hot and hat, but not hit
^	Represents any character not in the brackets	h[^oa]t finds hit, but not hot and hat
-	Represents any single character within the specified range	c[a-b]t finds cat and cbt

Wildcards kan også brukes i kombinasjon med hverandre.

SQL IN Operator

IN lar deg spesifisere flere verdier i en **WHERE** klausul. **IN** er en shorthand for mange **OR** betingelser.

Syntaks:
SELECT column_name(s)
FROM table_name
WHERE column_name **IN** (value1, value2, ...);

SELECT column_name(s)
FROM table_name
WHERE column_name **IN** (**SELECT** STATEMENT);

For eksempel:

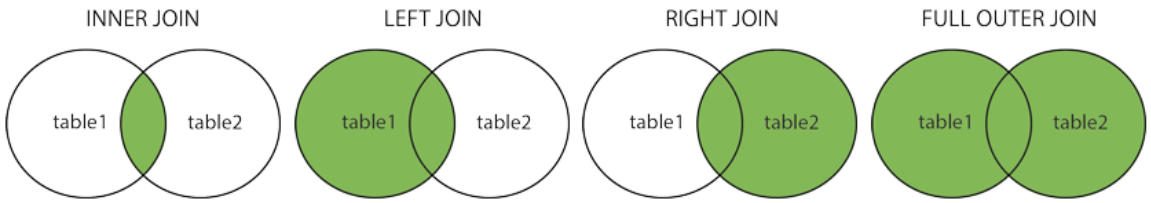
SELECT * FROM Customers
WHERE Country **IN** ('Germany', 'France', 'UK');

SELECT * FROM Customers
WHERE Country **IN** (**SELECT** Country **FROM** Suppliers);
Velger alle kunder som er fra samme land som suppliers

SQL BETWEEN Operator

BETWEEN operatoren velger verdier innen et gitt område. Verdiene kan være tall, tekst eller datoer. **BETWEEN** er inclusive, dvs at start og sluttverdier inngår.

Syntaks:

	SELECT column_name(s) **FROM** table_name **WHERE** column_name **BETWEEN** value1 **AND** value2; **Eksempel:** **SELECT * FROM** Products **WHERE** Price **BETWEEN** 10 **AND** 20;
SQL Aliases	SQL aliaser blir brukt til å gi tabeller, kolonner eller rader et midlertidig navn. Dette gjøres ofte for at resultatet i spørringene våre skal se mer ryddig og lesbart ut. Et alias eksisterer bare for lengden av spørringen. Vi lager alias med nøkkelordet **AS** **SELECT** CustomerID **AS** ID, CustomerName **AS** Customer **FROM** Customers; Aliaser er nyttige når: - Det er mer enn en tabell involvert i spørringen - Funksjoner er brukt i spørringen - Kolonnenavnet er stort og ikke særlig lesbart - To eller flere kolonner er kombinert sammen
SQL Joins	**JOIN** brukes til å slå sammen rader fra to eller flere tabeller basert på relasjonen mellom dem.  **<u>(INNER) JOIN</u>** returnerer poster som har matchende verdier i begge tabellene. **SELECT** column_name(s) **FROM** table1 **INNER JOIN** table2 **ON** table1.column_name = table2.column_name; OBS: **INNER JOIN** velger alle rader fra begge tabellene så lenge det er en match mellom kolonnene. Hvis det er poster som ikke matcher i begge tabellene vil ikke disse bli vist. **<u>LEFT (OUTER) JOIN</u>** returnerer alle poster fra venstre tabell og de matchende postene fra høyre tabell. **SELECT** column_name(s) **FROM** table1 **LEFT JOIN** table2

	ON table1.column_name = table2.column_name; **<u>RIGHT (OUTER) JOIN</u>** returnerer alle poster fra høyre tabell og de matchende postene fra høyre tabell **SELECT** column_name(s) **FROM** table1 **RIGHT JOIN** table2 **ON** table1.column_name = table2.column_name; **<u>FULL (OUTER) JOIN</u>** returnerer alle poster når det er en match i enten venstre eller høyre tabell. **SELECT** column_name(s) **FROM** table1 **FULL OUTER JOIN** table2 **ON** table1.column_name = table2.column_name **WHERE** condition; OBS: **FULL (OUTER) JOIN** kan føre til at man får returnert veldig store resultatsett. **<u>Self join</u>** er å joine en tabell med seg selv.
SQL UNION Operator	**UNION** operatoren brukes til å kombinere resultatsettet til to eller flere **SELECT** uttalelser. – Alle **SELECT** uttaleleser innen **UNION** må ha samme antall kolonner – Kolonnene må ha samme type data – Kolonnene i alle **SELECT** uttalelsene må også være i samme rekkefølge **SELECT** column_name(s) **FROM** table1 **UNION** **SELECT** column_name(s) **FROM** table2; **UNION** operatoren velger alltid ut distinkte verdier by default. For å få duplikate verdier må vi bruke **UNION ALL**
SQL GROUP BY Statement	**GROUP BY** uttalelsen grupperer rader som har samme verdier i oppsummeringsrader, som for eksempel “find the number of customers in each country”. **GROUP BY** brukes ofte sammen med aggregerte funksjoner (**`COUNT()`**, **`MAX()`**, **`MIN()`**, **`SUM()`**, **`AVG()`**) til å gruppere resultatsettet med en eller flere kolonner. **SELECT** column_name(s) **FROM** table_name **WHERE** condition **GROUP BY** column_name(s) **ORDER BY** column_name(s);

	Eks: **SELECT COUNT**(CustomerID), Country **FROM** Customers **GROUP BY** Country;
SQL HAVING Clause	**HAVING** klausulen ble lagt til i SQL fordi **WHERE** nøkkelordet ikke kan brukes med aggregerte funksjoner. **SELECT** column_name(s) **FROM** table_name **WHERE** condition **GROUP BY** column_name(s) **HAVING** condition **ORDER BY** column_name(s); Eks: **SELECT COUNT**(CustomerID), Country **FROM** Customers **GROUP BY** Country **HAVING COUNT**(CustomerID) > 5;
SQL EXISTS Operator	**EXISTS** operatoren blir brukt til å teste for om det eksisterer poster i en underspørring. **EXISTS** operatoren returnerer **TRUE** om underspørringen returnerer en eller flere poster. **SELECT** column_name(s) **FROM** table_name **WHERE EXISTS** (**SELECT** column_name **FROM** table_name **WHERE** condition); Eks: **SELECT** SupplierName **FROM** Suppliers **WHERE EXISTS** (**SELECT** ProductName **FROM** Products **WHERE** Products.SupplierID = Suppliers.supplierID **AND** Price < 20);
SQL ANY and ALL Operators	**ANY** og **ALL** operatorene gjør at du kan gjøre en sammenlikning mellom en enkelt kolonneverdi og en rekke av andre verdier. **ANY** operatoren: – Returnerer en boolsk verdi som resultat – Returnerer TRUE hvis noen av underspørringenes verdi møter betingelsene. **ANY** betyr at betingelsen er sann hvis operatoren er sann for noen av verdiene i området. **SELECT** ProductName **FROM** Products

	WHERE ProductID = **ANY** (**SELECT** ProductID **FROM** OrderDetails **WHERE** Quantity = 10); **ALL** operatoren: – Returnerer en boolsk verdi som resultat – Returnerer TRUE hvis alle underspørringenes verdi møter betingelsene – Brukes med **SELECT**, **WHERE** og **HAVING** uttrykk **ALL** betyr at betingelsen er sann hvis operatoren er sann for alle verdiene i området **SELECT ALL** ProductName **FROM** Products **WHERE** TRUE;
SQL SELECT INTO Statement	**SELECT INTO** uttrykket kopierer data fra en tabell til en ny tabell. **SELECT** * **INTO** newtable [**IN** externaldb] **FROM** oldtable **WHERE** condition; Eks: **SELECT * INTO** CustomersGermany **FROM** Customers **WHERE** Country = 'Germany'; **SELECT INTO** kan også brukes til å kopiere formatet til en tabell og lage en ny tabell med det samme formatet ved å bruke en **WHERE** betingelse som sørger for å returnere ingen data. Eks: **SELECT * INTO** newtable **FROM** oldtable **WHERE** 1 = 0;
SQL INSERT INTO SELECT Statement	**INSERT INTO SELECT** uttrykk kopierer data fra en tabell og legger det til en annen tabell. **INSERT INTO SELECT** uttrykket krever at datatypen i kilden og målet matcher. **INSERT INTO** table2 **SELECT * FROM** table1 **WHERE** condition;

SQL CASE Statement	**CASE** uttrykk går gjennom betingelsene og returnerer en verdi når den første betingelsen er møtt (som en if-then-else). Så fort en betingelse er møtt vil den stoppe og returnere resultatet. Hvis ingen betingelser er møtt returnerer den verdien i en **ELSE** klausul. Hvis det ikke er noen **ELSE** returnerer den **NULL**
	
	CASE
	  WHEN condition1 **THEN** result1
	  WHEN condition2 **THEN** result2
	  WHEN conditionN **THEN** resultN
	  ELSE result
	END;

SQL NULL Functions	Vi har ulike NULL funksjoner for å hente data fra kolonner som kan inneholde mye NULL data. Fordi **SELECT** vil kunne gi oss utelukkende NULL fordi en kolonne inneholder noen NULL verdier. Vi kan bruke **IFNULL()** eller **COALESCE()** funksjonene for å spesifisere at hvis en post er NULL vil den returnere en verdi vi spesifiserer, feks 0. **SELECT** ProductName, UnitPrice * (UnitsInStock + **IFNULL**(UnitsOnOrder, 0)) **FROM** Products; Eller **SELECT** ProductName, UnitPrice * (UnitsInStock + **COALESCE**(UnitsOnOrder, 0)) **FROM** Products; OBS: SQL Server bruker **ISNULL()**, MS ACCESS bruker **IsNull()** og Oracle bruker **NVL()**
SQL Stored Procedures for SQL Server	En **stored procedure** er en forberedt SQL kode som man kan lagre for å gjenbruke koden igjen. Hvis man har en SQL spørring man bruker mange ganger kan man lagre den som en **stored procedure** og kalle den for å utføre den. **CREATE PROCEDURE** procedure_name **AS** sql_statement **GO**; **EXEC** procedure_name; En stored procedure kan ha en eller flere parametere.
SQL Comments	Single line comment bruker to streker — All tekst mellom — og slutten av linja vil bli ignorert. Multi line comments bruker /* */ All tekst mellom /* og */ blir ignorert.

SQL Operators

SQL Arithmetic Operators

Operator	Description
+	Add
-	Subtract
*	Multiply
/	Divide
%	Modulo

SQL Bitwise Operators

Operator	Description
&	Bitwise AND
|	Bitwise OR
^	Bitwise exclusive OR

SQL Comparison Operators

Operator	Description
=	Equal to
>	Greater than
<	Less than
>=	Greater than or equal to
<=	Less than or equal to
<>	Not equal to

SQL Compound Operators

Operator	Description
+=	Add equals
-=	Subtract equals
*=	Multiply equals
/=	Divide equals
%=	Modulo equals
&=	Bitwise AND equals
^-=	Bitwise exclusive equals
|*=	Bitwise OR equals

SQL Logical Operators

Operator	Description
ALL	TRUE if all of the subquery values meet the condition
AND	TRUE if all the conditions separated by AND is TRUE
ANY	TRUE if any of the subquery values meet the condition
BETWEEN	TRUE if the operand is within the range of comparisons
EXISTS	TRUE if the subquery returns one or more records
IN	TRUE if the operand is equal to one of a list of expressions
LIKE	TRUE if the operand matches a pattern
NOT	Displays a record if the condition(s) is NOT TRUE
OR	TRUE if any of the conditions separated by OR is TRUE
SOME	TRUE if any of the subquery values meet the condition

SQL Database

https://www.w3schools.com/sql/

Tema	Notater
SQL CREATE DATABASE Statement	**CREATE DATABASE** brukes til å opprette en ny SQL database. Eks: **CREATE DATABASE** databasename; **OBS:** Man må ha admin rettigheter for å kunne opprette en ny database. Når man har opprettet en database kan man se oversikt ved hjelp av **SHOW DATABASES;**
SQL DROP DATABASE Statement	**DROP DATABASE** brukes til å slette en eksisterende database. Eks: **DROP DATABASE** databasename; **OBS:** Vær forsiktig med å slette en database! Det kan føre til tap av informasjon lagret i en database. Når man sletter en database må man ha adminrettigheter. Man kan sjekke hvilke databaser man har ved hjelp av **SHOW DATABASES;**
SQL BACKUP DATABASE for SQL Server	**BACKUP DATABASE** brukes til å ta full backup av en SQL database. Eks: **BACKUP DATABASE** databasename **TO DISK** = 'filepath'; Man kan ta backup av kun endringene som er gjort siden forrige backup: **BACKUP DATABASE** databasename **TO DISK** = 'filepath' **WITH DIFFERENTIAL;** **Tips:** Lag alltid backup av databasen et annet sted enn der selve databasen ligger, slik at man har en backup ved feks en diskcrash. Ved å bruke en **DIFFERENTIAL** backup sparer man tid.
SQL CREATE TABLE Statement	**CREATE TABLE** brukes til å lage en ny tabell i en database. Eks: Denne oppretter tabellen Persons med kolonnene PersonID, LastName, FirstName, Adress og City. **CREATE TABLE** Persons (PersonID int, LastName varchar(255), FirstName varchar(255), Address varchar(255), City varchar(255));

	Kolonneparameteret spesifiserer hvor mange kolonner vi vil ha. Deretter spesifiserer vi hva slags datatype dette er. Noen av de vanligste: char(størrelse), varchar(størrelse), int(størrelse), double(størrelse.desimaler) Alle datatyper her: https://www.w3schools.com/mysql/mysql_datatypes.asp Man kan legge inn verdier i tabellen ved hjelp av **INSERT INTO**
SQL DROP TABLE Statement	**DROP TABLE** brukes til å slette en eksisterende tabell i databasen. Det er viktig å være forsiktig når man sletter en tabell, for all informasjon i den blir permanent slettet. Syntaks: **DROP TABLE** table_name; **TRUNCATE TABLE** brukes til å slette all data inne i tabellen, men ikke selve tabellen. Syntaks: **TRUNCATE TABLE** table_name;
SQL ALTER TABLE Statement	**ALTER TABLE** brukes til å legge til, slette eller modifisere eksisterende tabeller. Den brukes også til å legge til eller fjerne ulike begrensninger. **ADD:** **ALTER TABLE** table_name **ADD** column_name datatype; **DROP COLUMN:** **ALTER TABLE** table_name **DROP COLUMN** column_name; **ALTER/MODIFY:** MYSQL / Oracle (før v.10G) **ALTER TABLE** table_name **MODIFY COLUMN** column_name datatype; SQL SERVER/MS ACCESS: **ALTER TABLE** table_name **ALTER COLUMN** column_name datatype; Oracle 10G og senere: **ALTER TABLE** table_name **MODIFY** column_name datatype;
SQL Constraints	SQL constraints brukes til å spesifisere regler for data i en tabell. Constraints brukes til å begrense hvilken type data vi kan bruke i en tabell. Det sikrer nøyaktigheten og påliteligheten til dataen i tabellen. Hvis det er brudd mellom begrensningen og datahandlingen, avbrytes handlingen. Begrensningene kan være på kolonnenivå eller tabellnivå. Kolonnenivåbegrensninger gjelder for en kolonne, og tabellnivåbegrensninger gjelder for hele tabellen.

	The following constraints are commonly used in SQL: • `NOT NULL` - Ensures that a column cannot have a NULL value • `UNIQUE` - Ensures that all values in a column are different • `PRIMARY KEY` - A combination of a `NOT NULL` and `UNIQUE`. Uniquely identifies each row in a table • `FOREIGN KEY` - Prevents actions that would destroy links between tables • `CHECK` - Ensures that the values in a column satisfies a specific condition • `DEFAULT` - Sets a default value for a column if no value is specified • `CREATE INDEX` - Used to create and retrieve data from the database very quickly Syntaks: **CREATE TABLE** table_name (column1 datatype **constraint**, column2 datatype **constraint**, column3 datatype **constraint**, ...);
SQL NOT NULL Constraint	Som standard kan en kolonne inneholde NULL verdier. **NOT NULL** constrainten sørger for at en kolonne ikke aksepterer NULL verdier. **<u>NOT NULL</u>** <u>med</u> **<u>CREATE TABLE</u>** Den følgende setningen sørger for at ID, LastName og FirstName kolonnene ikke aksepterer NULL verdier når Persons tabellen blir oppretta: **CREATE TABLE** Persons (ID int **NOT NULL**, LastName varchar(255) **NOT NULL**, FirstName varchar(255) **NOT NULL**, Age int); **<u>NOT NULL</u>** <u>med</u> **<u>ALTER TABLE</u>** For å lage en NOT NULL constraint på Age kolonnen når Persons tabellen allerede er opptretta kan man skrive følgende: **ALTER TABLE** Persons **MODIFY** Age int **NOT NULL**;
SQL UNIQUE Constraint	**UNIQUE** er en constraint som sørger for at alle verdiene i en kolonne er ulike. Både **UNIQUE** og **PRIMARY KEY** constraints sørger for dette. En **PRIMARY KEY** har automatisk en **UNIQUE** constraint. MEN, man kan bare ha EN **PRIMARY KEY** i en tabell, mens man kan ha mange **UNIQUE**. **<u>MySQL syntaks for CREATE TABLE:</u>** **CREATE TABLE** Persons (ID int **NOT NULL**, LastName varchar(255) **NOT NULL**, FirstName varchar(255), Age int, **UNIQUE** (ID)); (OBS SQL Server, Oracle og MS Access bruker UNIQUE etter den definerte kolonnen)

	<u>Opprette flere UNIQUE kolonner:</u> **CREATE TABLE** Persons (ID int **NOT NULL**, LastName varchar(255) **NOT NULL**, FirstName varchar(255), Age int, **CONSTRAINT UC**_Person **UNIQUE** (ID,LastName)); **<u>MySQL syntaks for ALTER TABLE:</u>** **ALTER TABLE** Persons **ADD UNIQUE** (ID); For multiple columns: **ALTER TABLE** Persons **ADD CONSTRAINT** UC_Person **UNIQUE** (ID,LastName); **<u>MySQL syntaks for DROP TABLE:</u>** **ALTER TABLE** Persons **DROP INDEX UC**_Person; OBS: for SQL Server, Oracle og MS Access bruker **DROP CONSTRAINT UC**_Person;
SQL PRIMARY KEY Constraint	**PRIMARY KEY** constrainten brukes til å definere en unik token i tabellen. For å ha en relasjonsdatabase knytter man **PRIMARY KEY** med **FOREIGN KEY**. Det som er en **PRIMARY KEY** i en tabell knytter en annen tabell ved å bruke den som referanse i en annen tabell. **PRIMARY KEYS** MÅ være **UNIQUE** og kan ikke være NULL. En tabell kan kun ha EN **PRIMARY KEY**, men den kan bestå av en eller flere kolonner (felter). **CREATE TABLE** Persons (ID int **NOT NULL**, LastName varchar(255) **NOT NULL**, FirstName varchar(255), Age int, **PRIMARY KEY** (ID)); SQL Server, Oracle og MS Access bruker: ID int **NOT NULL PRIMARY KEY**, **<u>For flere kolonner:</u>** **CREATE TABLE** Persons (ID int **NOT NULL**, LastName varchar(255) **NOT NULL**, FirstName varchar(255), Age int, **CONSTRAINT PK**_Person **PRIMARY KEY** (ID,LastName)); MERK: I eksempelet over er det kun en **PRIMARY KEY** (PK_Person), men verdien av nøkkelen består av to kolonner (ID og LastName) **<u>PRIMARY KEY med ALTER TABLE:</u>**

For å lage en PK constraint når man først har opprettet en tabell
ALTER TABLE Persons
ADD PRIMARY KEY (ID);

For flere kolonner:
ALTER TABLE Persons
ADD CONSTRAINT PK_Person **PRIMARY KEY** (ID,LastName);
MERK: For å legge til PK i etterkant MÅ kolonnene være definert som NOT NULL

PRIMARY KEY med DROP TABLE:
ALTER TABLE Persons
DROP PRIMARY KEY;

OBS for SQL Server, Oracle og MS Access bruker man:
DROP CONSTRAINT PK_Person;

SQL FOREIGN KEY Constraint

FOREIGN KEY constrainten brukes til å forhindre input som ødelegger linken mellom to eller flere tabeller.
En **FOREIGN KEY** er en **PRIMARY KEY** fra en annen tabell og brukes til å knytte tabellene sammen i en relasjonsdatabase.

Tabellen med FK kalles child table og tabellen som har den som PK blir kalt parent table.

Persons Table

PersonID	LastName	FirstName	Age
1	Hansen	Ola	30
2	Svendson	Tove	23
3	Pettersen	Kari	20

Orders Table

OrderID	OrderNumber	PersonID
1	77895	3
2	44678	3
3	22456	2
4	24562	1

I tabellen over er PK markert med gult og FK markert med grønn. PersonID er PK i Persons tabellen og blir FK i Orders tabellen. Dette knytter tabellene sammen. I Orders tabellen er OrderID PK.

CREATE TABLE Orders (
  OrderID int **NOT NULL**,
  OrderNumber int **NOT NULL**,
  PersonID int,
  PRIMARY KEY (OrderID),
  FOREIGN KEY (PersonID) **REFERENCES** Persons(PersonID)
);

I SQL Server, Oracle og MS Access brukes:
CREATE TABLE Orders (
  OrderID int **NOT NULL PRIMARY KEY**,
  OrderNumber int **NOT NULL**,
  PersonID int **FOREIGN KEY REFERENCES** Persons(PersonID)

	); **For flere FOREIGN KEYS:** **CREATE TABLE** Orders (OrderID int **NOT NULL**, OrderNumber int **NOT NULL**, PersonID int, **PRIMARY KEY** (OrderID), **CONSTRAINT FK**_PersonOrder **FOREIGN KEY** (PersonID) **REFERENCES** Persons(PersonID)); **<u>FOREIGN KEY med ALTER TABLE</u>** Når man skal endre en oppretta tabell bruker man **ALTER TABLE** Orders **ADD FOREIGN KEY** (PersonID) **REFERENCES** Persons(PersonID); For å endre flere FK bruker man **ALTER TABLE** Orders **ADD CONSTRAINT FK**_PersonOrder **FOREIGN KEY** (PersonID) **REFERENCES** Persons(PersonID); **<u>DROP FOREIGN KEY</u>** **ALTER TABLE** Orders **DROP FOREIGN KEY FK**_PersonOrder; I SQL Server, Oracle og MS Access bruker man **DROP CONSTRAINT FK**_PersonOrder;
SQL CHECK Constraint	**CHECK** constrainten brukes til å sette grenseverdier for kolonner. Hvis du definerer **CHECK** på en kolonne vil man kun sette et definert verdiområde i kolonna. Den kan også defineres ut fra verdier i andre kolonner i raden. Feks at alder må være over 18: **CREATE TABLE** Persons (ID int **NOT NULL**, LastName varchar(255) **NOT NULL**, FirstName varchar(255), Age int, **CHECK** (Age>=18)); SQL Server, Oracle og MS Access bruker Age int **CHECK** (Age>=18) **For flere kolonner:** **CREATE TABLE** Persons (ID int **NOT NULL**, LastName varchar(255) **NOT NULL**, FirstName varchar(255), Age int, City varchar(255), **CONSTRAINT CHK**_Person **CHECK** (Age>=18 AND City='Sandnes')); **<u>Bruke CHECK med ALTER TABLE</u>** **ALTER TABLE** Persons **ADD CHECK** (Age>=18);

	For flere kolonner: **ALTER TABLE** Persons **ADD CONSTRAINT CHK**_PersonAge **CHECK** (Age>=18 **AND** City='Sandnes'); **<u>Bruk CHECK med DROP TABLE</u>** **ALTER TABLE** Persons **DROP CHECK CHK**_PersonAge; SQL Server, Oracle og MS Access bruker **DROP CONSTRAINT CHK**_PersonAge;
SQL DEFAULT Constraint	**DEFAULT** constrainten brukes til å sette default verdier for kolonner. Default verdien blir lagt til alle nye poster med mindre annet er spesifisert. **<u>DEFAULT med CREATE TABLE</u>** **CREATE TABLE** Persons (ID int **NOT NULL**, LastName varchar(255) **NOT NULL**, FirstName varchar(255), Age int, City varchar(255) **DEFAULT** 'Sandnes'); I dette eksempelet vil det stå Sandnes i alle postene i City kolonnen med mindre man spesifiserer noe annet når man legger inn rader. Man kan også bruke **DEFAULT** til system values som for eksempel funksjonen **GETDATE()**; **CREATE TABLE** Orders (ID int **NOT NULL**, OrderNumber int **NOT NULL**, OrderDate date **DEFAULT GETDATE()**); **<u>DEFAULT med ALTER TABLE</u>** **ALTER TABLE** Persons **ALTER** City **SET DEFAULT** 'Sandnes'; OBS: Annen syntaks for SQL Server, Oracle og MS Access **<u>DEFALUT med DROP TABLE</u>** **ALTER TABLE** Persons **ALTER** City **DROP DEFAULT**; SQL Server, Oracle og MS Access bruker: **ALTER COLUMN** City **DROP DEFAULT**;
SQL CREATE INDEX Statement	**CREATE INDEX** brukes til å opprette indekser i tabeller. Indekser brukes til å hente data raskere fra en database. Brukere kan ikke se indekser, de blir brukt til å gjøre søk/query raskere. OBS: Å oppdatere tabeller med indekser tar lenger tid enn uten, fordi indeksene også må oppdateres. Derfor er det lurt å bare bruke indeks når det er noe man ofte søker etter. **CREATE INDEX** Oppretter indeks på en tabell. Man kan ha duplikate verdier **CREATE INDEX** index_name

	ON table_name (column1, column2, ...); **CREATE UNIQUE INDEX** Oppretter en unik indeks på en tabell. Duplikate verdier er ikke tillat. **CREATE UNIQUE INDEX** index_name **ON** table_name (column1, column2, ...); **DROP INDEX** Brukes til å slette en indeks i en tabell **ALTER TABLE** table_name **DROP INDEX** index_name; OBS: Drop har annen syntaks i SQL Server, Oracle og MS Access
SQL AUTO INCREMENT Field	**AUTO_INCREMENT** brukes til å generere et unikt nummer automatisk når en ny post legges til i tabellen. Man bruker ofte dette sammen med **PRIMARY KEY** for å automatisk opprette unike nummer. **CREATE TABLE** Persons (Personid int **NOT NULL AUTO_INCREMENT**, LastName varchar(255) **NOT NULL**, FirstName varchar(255), Age int, **PRIMARY KEY** (Personid)); By default starter **AUTO_INCREMENT** på 1 og øker med 1 av gangen. Hvis man ønsker å starte den fra en annen verdi bruker man: **ALTER TABLE** Persons **AUTO_INCREMENT**=100; OBS: Det er annen syntaks SQL Server, Oracle og MS Access
SQL Working With Dates	Det vanskeligste med å bruke datoer i SQL er å sørge for at datoformatet blir riktig og matcher datoformatet som er satt i kolonnen. SQL Date Data Types **MySQL** comes with the following data types for storing a date or a date/time value in the database: • DATE - format YYYY-MM-DD • DATETIME - format: YYYY-MM-DD HH:MI:SS • TIMESTAMP - format: YYYY-MM-DD HH:MI:SS • YEAR - format YYYY or YY Dato typen blir valgt for kolonnen når man oppretter tabellen Dato kan enkelt sammenliknes hvis tid ikke er involvert. Ikke bruk tid hvis man ikke må! **SELECT * FROM** Orders **WHERE** OrderDate='2008-11-11'
SQL Views	View er en virtuell tabell som er basert på resultatsetta i SQL uttrykket ditt. Altså det du ser når du kjører SQL setninger. Man kan spesifisere at man ønsker å få tilbake et view med **CREATE VIEW** Man kan oppdatere view med **CREATE OR REPLACE VIEW**. Man kan slette en view med **DROP VIEW**.

SQL Injection	SQL injection er en kode injection teknikk som kan ødelegge databaser. SQL injection er en av de vanligste måtene å web hacke. SQL injection handler om å sette inn ondsinnet kode i en SQL statemenet via web.
	
	SQL injections skjer ofte når man ber brukeren skal komme med input på en side, som feks brukernavn og passord. Istedenfor å fylle inn informasjon sender man inn SQL statements som ødelegger databasen uten at man vet det.
	
	For eksempel at en bruker sender inn følgende i et skjema:
	txtUserId = getRequestString("UserId");
	txtSQL = "SELECT * FROM Users WHERE UserId = " + txtUserId;
	
	1=1 is always true
	Et vanlig eksempel er å sende inn en statement med 1=1, noe som alltid er sant.
	
	UserId: 105 OR 1=1
	
	I dette eksempelet blir det kjørt følgende statement:
	SELECT * FROM Users **WHERE** UserId = 105 OR 1=1;
	Siden 1=1 alltid er sann vil dette føre til at man får ut alle brukere.
	
	SELECT UserId, Name, Password **FROM** Users **WHERE** UserId = 105 or 1=1;
	
	“=” is always true
	På samme måte som over kan en bruker skrive inn “ or””=” som vil føre til:
	SELECT * FROM Users **WHERE** Name ="" or ""="" **AND** Pass ="" or ""=""
	Dette er always true og vil kunne gi info om alle brukere
	
	Batched SQL statements
	De fleste databaser støtter batch statements, altså to eller flere statements separert med semikolon.
	
	Eks:
	txtUserId = getRequestString("UserId");
	txtSQL = "SELECT * FROM Users WHERE UserId = " + txtUserId;
	
	Også denne inputen:
	
	User id: 105; DROP TABLE Suppliers
	
	Fører til:
	SELECT * FROM Users **WHERE** UserId = 105; **DROP TABLE** Suppliers;
	
	Beskytte mot injections
	Man kan for det første sørge for at webskjemaet sjekker at man skriver riktig input. Man kan også beskytte i SQL ved hjelp av SQL parametere.
	Man kan bruke @ sammen med parameter for å sørge for at hvert parameter er korrekt for sin kolonne og at den blir tolket bokstavelig og ikke som en del av en SQL setning som skal kjøres.

SQL Hosting	Hvis man ønsker at en webside skal lagre eller hente data fra en database bør websiden ha tilgang til et databasesystem som bruker SQL språket. MySQL er den vanligste SQL hosten. Andre populære er MS SQL Server, Oracle og MS Access. MS SQL Server og Oracle er kraftfulle og robuste SQL-systemer, men de kan være dyre. MySQL er den mest populære, mye fordi den er billig, kraftfull og robust. MS Access er et alternativ for simple databaser, men er ikke optimalisert for høy trafikk.

SQL Data Types

String Data Types

Data type	Description
CHAR(size)	A FIXED length string (can contain letters, numbers, and special characters). The *size* parameter specifies the column length in characters - can be from 0 to 255. Default is 1
VARCHAR(size)	A VARIABLE length string (can contain letters, numbers, and special characters). The *size* parameter specifies the maximum column length in characters - can be from 0 to 65535
BINARY(size)	Equal to CHAR(), but stores binary byte strings. The *size* parameter specifies the column length in bytes. Default is 1
VARBINARY(size)	Equal to VARCHAR(), but stores binary byte strings. The *size* parameter specifies the maximum column length in bytes.
TINYBLOB	For BLOBs (Binary Large Objects). Max length: 255 bytes
TINYTEXT	Holds a string with a maximum length of 255 characters
TEXT(size)	Holds a string with a maximum length of 65,535 bytes
BLOB(size)	For BLOBs (Binary Large Objects). Holds up to 65,535 bytes of data
MEDIUMTEXT	Holds a string with a maximum length of 16,777,215 characters
MEDIUMBLOB	For BLOBs (Binary Large Objects). Holds up to 16,777,215 bytes of data
LONGTEXT	Holds a string with a maximum length of 4,294,967,295 characters
LONGBLOB	For BLOBs (Binary Large Objects). Holds up to 4,294,967,295 bytes of data
ENUM(val1, val2, val3, ...)	A string object that can have only one value, chosen from a list of possible values. You can list up to 65535 values in an ENUM list. If a value is inserted that is not in the list, a blank value will be inserted. The values are sorted in the order you enter them
SET(val1, val2, val3, ...)	A string object that can have 0 or more values, chosen from a list of possible values. You can list up to 64 values in a SET list

Numeric Data Types

Data type	Description
BIT(*size*)	A bit-value type. The number of bits per value is specified in *size*. The *size* parameter can hold a value from 1 to 64. The default value for *size* is 1.
TINYINT(*size*)	A very small integer. Signed range is from -128 to 127. Unsigned range is from 0 to 255. The *size* parameter specifies the maximum display width (which is 255)
BOOL	Zero is considered as false, nonzero values are considered as true.
BOOLEAN	Equal to BOOL
SMALLINT(*size*)	A small integer. Signed range is from -32768 to 32767. Unsigned range is from 0 to 65535. The *size* parameter specifies the maximum display width (which is 255)
MEDIUMINT(*size*)	A medium integer. Signed range is from -8388608 to 8388607. Unsigned range is from 0 to 16777215. The *size* parameter specifies the maximum display width (which is 255)
INT(*size*)	A medium integer. Signed range is from -2147483648 to 2147483647. Unsigned range is from 0 to 4294967295. The *size* parameter specifies the maximum display width (which is 255)
INTEGER(*size*)	Equal to INT(size)
BIGINT(*size*)	A large integer. Signed range is from -9223372036854775808 to 9223372036854775807. Unsigned range is from 0 to 18446744073709551615. The *size* parameter specifies the maximum display width (which is 255)
FLOAT(*size*, *d*)	A floating point number. The total number of digits is specified in *size*. The number of digits after the decimal point is specified in the *d* parameter. This syntax is deprecated in MySQL 8.0.17, and it will be removed in future MySQL versions
FLOAT(*p*)	A floating point number. MySQL uses the *p* value to determine whether to use FLOAT or DOUBLE for the resulting data type. If *p* is from 0 to 24, the data type becomes FLOAT(). If *p* is from 25 to 53, the data type becomes DOUBLE()
DOUBLE(*size*, *d*)	A normal-size floating point number. The total number of digits is specified in *size*. The number of digits after the decimal point is specified in the *d* parameter
DOUBLE PRECISION(*size*, *d*)	
DECIMAL(*size*, *d*)	An exact fixed-point number. The total number of digits is specified in *size*. The number of digits after the decimal point is specified in the *d* parameter. The maximum number for *size* is 65. The maximum number for *d* is 30. The default value for *size* is 10. The default value for *d* is 0.
DEC(*size*, *d*)	Equal to DECIMAL(size,d)

Note: All the numeric data types may have an extra option: UNSIGNED or ZEROFILL. If you add the UNSIGNED option, MySQL disallows negative values for the column. If you add the ZEROFILL option, MySQL automatically also adds the UNSIGNED attribute to the column.

Date and Time Data Types

Data type	Description
DATE	A date. Format: YYYY-MM-DD. The supported range is from '1000-01-01' to '9999-12-31'
DATETIME(*fsp*)	A date and time combination. Format: YYYY-MM-DD hh:mm:ss. The supported range is from '1000-01-01 00:00:00' to '9999-12-31 23:59:59'. Adding DEFAULT and ON UPDATE in the column definition to get automatic initialization and updating to the current date and time
TIMESTAMP(*fsp*)	A timestamp. TIMESTAMP values are stored as the number of seconds since the Unix epoch ('1970-01-01 00:00:00' UTC). Format: YYYY-MM-DD hh:mm:ss. The supported range is from '1970-01-01 00:00:01' UTC to '2038-01-09 03:14:07' UTC. Automatic initialization and updating to the current date and time can be specified using DEFAULT CURRENT_TIMESTAMP and ON UPDATE CURRENT_TIMESTAMP in the column definition
TIME(*fsp*)	A time. Format: hh:mm:ss. The supported range is from '-838:59:59' to '838:59:59'
YEAR	A year in four-digit format. Values allowed in four-digit format: 1901 to 2155, and 0000. MySQL 8.0 does not support year in two-digit format.

SQL Server Data Types

String Data Types

Data type	Description	Max size	Storage
char(n)	Fixed width character string	8,000 characters	Defined width
varchar(n)	Variable width character string	8,000 characters	2 bytes + number of chars
varchar(max)	Variable width character string	1,073,741,824 characters	2 bytes + number of chars
text	Variable width character string	2GB of text data	4 bytes + number of chars
nchar	Fixed width Unicode string	4,000 characters	Defined width x 2
nvarchar	Variable width Unicode string	4,000 characters	
nvarchar(max)	Variable width Unicode string	536,870,912 characters	
ntext	Variable width Unicode string	2GB of text data	
binary(n)	Fixed width binary string	8,000 bytes	
varbinary	Variable width binary string	8,000 bytes	
varbinary(max)	Variable width binary string	2GB	
image	Variable width binary string	2GB	

Numeric Data Types

Data type	Description	Storage
bit	Integer that can be 0, 1, or NULL	
tinyint	Allows whole numbers from 0 to 255	1 byte
smallint	Allows whole numbers between -32,768 and 32,767	2 bytes
int	Allows whole numbers between -2,147,483,648 and 2,147,483,647	4 bytes
bigint	Allows whole numbers between -9,223,372,036,854,775,808 and 9,223,372,036,854,775,807	8 bytes
decimal(p,s)	Fixed precision and scale numbers. Allows numbers from -10^38 +1 to 10^38 –1. The p parameter indicates the maximum total number of digits that can be stored (both to the left and to the right of the decimal point). p must be a value from 1 to 38. Default is 18. The s parameter indicates the maximum number of digits stored to the right of the decimal point. s must be a value from 0 to p. Default value is 0	5-17 bytes
numeric(p,s)	Fixed precision and scale numbers. Allows numbers from -10^38 +1 to 10^38 –1. The p parameter indicates the maximum total number of digits that can be stored (both to the left and to the right of the decimal point). p must be a value from 1 to 38. Default is 18. The s parameter indicates the maximum number of digits stored to the right of the decimal point. s must be a value from 0 to p. Default value is 0	5-17 bytes
smallmoney	Monetary data from -214,748.3648 to 214,748.3647	4 bytes
money	Monetary data from -922,337,203,685,477.5808 to 922,337,203,685,477.5807	8 bytes
float(n)	Floating precision number data from -1.79E + 308 to 1.79E + 308. The n parameter indicates whether the field should hold 4 or 8 bytes. float(24) holds a 4-byte field and float(53) holds an 8-byte field. Default value of n is 53.	4 or 8 bytes
real	Floating precision number data from -3.40E + 38 to 3.40E + 38	4 bytes

Date and Time Data Types

Data type	Description	Storage
datetime	From January 1, 1753 to December 31, 9999 with an accuracy of 3.33 milliseconds	8 bytes
datetime2	From January 1, 0001 to December 31, 9999 with an accuracy of 100 nanoseconds	6-8 bytes
smalldatetime	From January 1, 1900 to June 6, 2079 with an accuracy of 1 minute	4 bytes
date	Store a date only. From January 1, 0001 to December 31, 9999	3 bytes
time	Store a time only to an accuracy of 100 nanoseconds	3-5 bytes
datetimeoffset	The same as datetime2 with the addition of a time zone offset	8-10 bytes
timestamp	Stores a unique number that gets updated every time a row gets created or modified. The timestamp value is based upon an internal clock and does not correspond to real time. Each table may have only one timestamp variable	

Other Data Types

Data type	Description
sql_variant	Stores up to 8,000 bytes of data of various data types, except text, ntext, and timestamp
uniqueidentifier	Stores a globally unique identifier (GUID)
xml	Stores XML formatted data. Maximum 2GB
cursor	Stores a reference to a cursor used for database operations
table	Stores a result-set for later processing